DSLS Programming 5: Storing data in SQL Databases

Martijn Herber

Created: 2024-09-26 do 14:55

1. Lecture 3: SQL Databases

- How is this related to distributed programming?
  - Concurrency!
  - It's BIG DATA!
- At some point your distributed application needs to store data
- It's easy to handle concurrency/synchronization with a database
- Also: Spark heavily draws on SQL databases

2. Relational Databases

- Invented to store and search tabular data
- "Relational" means you split or separate data to reduce redundancy into "related" tables
- Standard interface is an English-like declarative language called "SQL"
- Offers safety for your data, and concurrent access ("ACID")

3. "CRUD" operations

- Create – new tables
- Read – data from the tables using SQL
- Update – alter tables or databases
- Delete – delete tables or whole databases

4. The ACID principles

- Atomicity
- Consistency
- Isolation
- Durability

5. Accessing a database from Python

- Python doesn't offer a way to access "real" remote DB's by default
  - (It does have a built-in database called "sqlite")
- We will use a module called `sqlalchemy` to connect and query with
- Technically it is an "Object Relational Mapper" (ORM)
- This means you can "save" Python objects to a DB, and retrieve them
- However, we can write "raw" SQL too, as we'll do here

6. Our database MySQL / MariaDB

- We have a central Database on the BIN network: MySQL / MariaDB
  - (It really has two names – long story)
- It serves many users concurrently including yourself
- It's reasonably powerful, heavily optimized for CRUD

7. Connecting

- You have an account; login info is in a file called `mysql_account.txt`
- The server is located at `mariadb.bin.bioinf.nl` and only accesible from the BIN network
- You have one (1) database on this server; for more, ask Peter/Marcel
  - (For this course you don't need more)

- Inside this database you can have as many tables as you like.

8. Table column types

- SQL tables have columns of particular enforced types – the schema
- "Enforced" means you get type errors if the data is of a different type
  - (Or can't be coerced)
- Most types are familiar to you from Python (Integers, text (string) etc.
- Changing types of columns is possible, but takes time

9. Table column types (2)

Type	Keyword	Storage
Number, integer	`INTEGER`	64 bit64
Floating point number	`DOUBLE`	128 bit
fixed-width text	`VARCHAR(x)`	x * 16 byte (unicode)xx
true/ false	`BOOLEAN`	1 byte
large(ish) text	`BLOB` or `TEXT`	64KB
dates	`DATETIME`	depends

10. Normalization

- Aims to reduce redundancy by "removing common elements"
- If you notice data/types which recur in your database, maybe you should pull them out into their own table
- …this is doubly true if you may need to **update** them separately
- You refer to other tables using `FOREIGN KEYS`
- It is sometimes hard to "pull apart" the data; in that case you may need to add columns (this is pretty cheap, storage-wise)

11. Normalization: Third Normal Form ("3NF")

- repeating groups in individual tables.
- a separate table for each set of related data.
- each set of related data with a primary key.
- Create separate tables for sets of values that apply to multiple records.
- Relate these tables with a foreign key.
- Eliminate fields that don't depend on the key.

12. SQL Basics

- SQL is declarative: you describe what you want, not how to do it
- It's just like English (or `pandas`) but more precise
- Fenna would call it "vectorized" ;-)
- There's (literally) millions of tutorials online (see assignment links)
- The general form:

```
VERB WHAT FROM DATABASE/TABLE WHERE CONDITION;
```

13. Connecting

- you can connect using the "mysql" command-line program (try it!)
- or you can use a programmatic, python approach

```
from sqlalchemy import engine, url, text
connectionstring = open("mysecrets_outside_git_directory.txt").read()
engine = engine.create_engine("mysql+mysqldb://YOURUSERNAME:YOURPASSWORDHERE@mariadb.bin.bioinf.nl/YOURDATABASENAME")
conn = engine.connect()
conn.execute("STATUS")
```

14. (Create)RUD

- You need some tables to work on; let's create one;

```
CREATE TABLE Proteins (
    id INTEGER AUTO_INCREMENT PRIMARY KEY,
    name TEXT NOT NULL,
    sequence TEXT NOT NULL
);
```

15. INSERTing some data

- It's still pretty empty here!
- We use SQL `INSERT` statements to put data into our table
- You need to match the type and number of columns (unless `DEFAULT`)
- To prevent SQL injection, we use a short hand form called "parameterization"

16. INSERTing some data (2)

```
from sqlalchemy.sql import text
query = text("INSERT INTO Proteins (name, sequence) VALUES (:name, :sequence)")
params = {"name" : "CCPA" , "sequence" : "MSNITIYDVAREANVSMATVSRVVNGNPNV"}
conn.execute(query, params)
```

17. SELECTing what you want (the "R" in CRUD)

```
SELECT * FROM Proteins WHERE id = 1;
SELECT sequence FROM Proteins WHERE
```

18. SELECTing with foreign keys

- If you normalize into different tables (good!) you need to alter your `SELECT`
- First define your tables to refer to each other;

```
CREATE TABLE Genes (
    id INTEGER AUTO_INCREMENT PRIMARY KEY,
    name TEXT NOT NULL,
    sequence TEXT NOT NULL,
    FOREIGN KEY (proteinid) REFERENCES Proteins(id)
);
```

19. SELECTing with foreign keys (2)

- You can `SELECT` on foreign keys directly, or combine tables using `JOIN`

```
SELECT Genes.name, Proteins.name FROM Genes INNER JOIN Proteins ON Genes.proteinid = Protein.id;
```

20. Joins across TABLEs

- There are multiple types of JOIN;
    1. `INNER JOIN` ; only rows with matching values in both tables
    2. `LEFT JOIN` and `RIGHT JOIN` ; all rows from left or right table, null from the other
    3. `FULL OUTER JOIN` ; (unsupported) but would be all rows from all tables
    4. `CROSS JOIN` ; cartesian product of the tables

21. UPDATING: the "U" in CRUD

- If data is already present, you can't `INSERT`
- Have to use `UPDATE` instead, but syntax is otherwise similar

```
UPDATE Proteins SET sequence = "M" WHERE id = 1;
DELETE FROM Proteins WHERE id = 1;
```

22. ALTERing and DELETEing ("D" in "CRUD")

- If you need to add or delete columns to an existing table, you can just do so:

```
ALTER TABLE Genes ADD coding BOOLEAN AFTER sequence;
```

- This can be very slow when you have a large existing table!

23. ALTERing and DELETEing ("D" in "CRUD")

- You can delete the whole table with

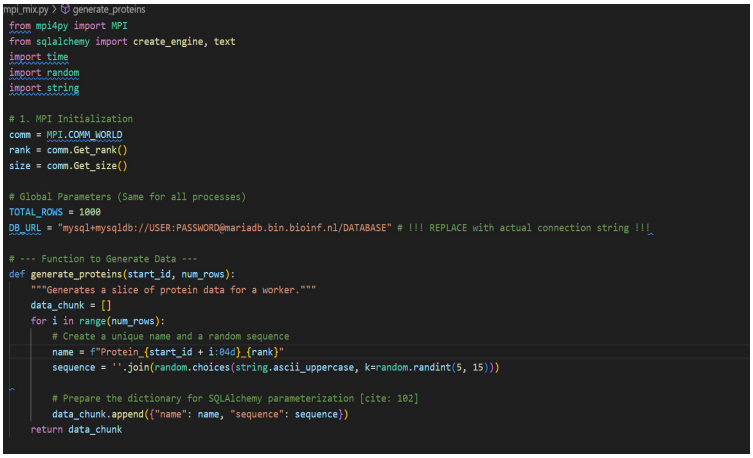
```
DROP TABLE IF EXISTS Proteins;
TRUNCATE TABLE Proteins;
DROP DATABASE yourdatabase;
```

24. What a database is good for

- Data can be very large, but not >terabytes (typically)
- Concurrent access is taken care of, and very efficient
- Data storage is very efficient
- Common operations (eg. searching with `SELECT`) are very fast

25. What a database is not good for

- Truly huge data
- Concurrent processing
- "Freeform" functions applied to the data (but: stored procedures)
- "Scientific" processing and machine learning



```python
# --- Main Logic ---

if __name__ == '__main__':
    start_time = time.time()

    # 2. Work Partitioning
    chunk_size = TOTAL_ROWS // size
    remainder = TOTAL_ROWS % size

    # Calculate the start row for this specific worker
    # (1 if rank < remainder else 0) :
    # a mathematical guarantee that only the exact number
    # of processes needed (equal to the remainder) get
    # the extra row, starting from the beginning of the worker list.
    my_rows = chunk_size + (1 if rank < remainder else 0)
    start_row = rank * chunk_size + min(rank, remainder)

    # --- MASTER PROCESS (Rank 0) Logic ---
    if rank == 0:
        print(f"--- Starting Distributed Database Load with {size} workers ---")

        # SQL Commands (Must be executed by a single process)
        CREATE_TABLE_SQL = text("""
            CREATE TABLE Proteins (
                id INTEGER AUTO_INCREMENT PRIMARY KEY,  -- Database guarantees unique IDs [cite: 90]
                name TEXT NOT NULL,
                sequence TEXT NOT NULL
            );
        """) # [cite: 89, 91, 92]

        DROP_TABLE_SQL = text("DROP TABLE IF EXISTS Proteins;") # [cite: 138]

        try:
            # Create the SQLAlchemy engine [cite: 81]
            engine = create_engine(DB_URL)
            with engine.connect() as conn:
                # Clean up previous table and create the new one
                conn.execute(DROP_TABLE_SQL)
                conn.execute(CREATE_TABLE_SQL)
                conn.commit()
            print(f"Master (Rank 0) created the 'Proteins' table.")
        except Exception as e:
            print(f"Master error during setup: {e}")
            comm.Abort() # Abort all processes if setup fails

    # Wait for the table creation to finish before any worker tries to insert
    # If a worker (like Rank 1) ran its insertion code before the master
    # (Rank 0) finished executing the CREATE TABLE command,
    # it would crash because the table wouldn't exist yet.
    # We need a mechanism to force every process to wait
    # until the master explicitly confirms that the table setup is complete.
    """
    Workers Wait: All the worker processes (Rank 1, 2, 3, etc.) execute quickly
    because they only run the initial setup code and then
    immediately reach the comm.Barrier(). They stop and wait there.

    Master Completes Setup: The Master process (Rank 0) takes longer because it
    executes the slow, essential tasks: connecting to the DB, running DROP TABLE,
    running CREATE TABLE, and running conn.commit().

    Synchronization: Once Rank 0 successfully finishes the setup and hits the
    comm.Barrier(), the barrier is released.

    Safe Proceeding: Now, every worker can safely proceed to its insertion code,
    guaranteed that the target Proteins table exists and is ready to accept data.
    """
    comm.Barrier()
```

```python
# --- ALL WORKERS (Including Rank 0) Logic ---


# 3. Data Generation and Insertion
if my_rows > 0: # we're using this condition instead of rank !=0 to allow rank 0 to also insert data
    # Generate the specific chunk of data for this worker
    my_data = generate_proteins(start_row, my_rows)


    # INSERT query using parameterization [cite: 102]
    INSERT_SQL = text("INSERT INTO Proteins (name, sequence) VALUES (:name, :sequence)")


    try:
        # Each worker establishes its own connection for concurrency [cite: 82]
        engine = create_engine(DB_URL)
        with engine.connect() as conn:
            # Execute a bulk insert using the list of dictionaries
            conn.execute(INSERT_SQL, my_data)
            conn.commit()
        print(f"Worker {rank:02d} inserted {len(my_data)} rows.")
    except Exception as e:
        print(f"Worker {rank:02d} error during insertion: {e}")
        # Do not abort the whole communicator here, as other workers might succeed

comm.Barrier() # Final synchronization point


if rank == 0:
    end_time = time.time()
    print(f"--- Total execution time for {TOTAL_ROWS} rows: {end_time - start_time:.4f} seconds ---")
```